

CURSO COMPLETO

SQL



PostgreSQL

MÓDULO 4

AGrupamentos

AGrupamentos

AGrupamentos

O QUE VEREMOS NESTE MÓDULO

MÓDULO 4 FUNÇÕES DE AGREGAÇÃO E AGRUPAMENTOS

01	COUNT	109
02	SUM	113
03	AVG, MIN e MAX	114
04	GROUP BY – Criando agrupamentos	115
05	GROUP BY, WHERE e HAVING – Filtros em agrupamentos	117
06	EXERCÍCIOS	120
07	GABARITO EXPLICATIVO	125

A função de agregação COUNT faz a contagem de valores de uma coluna.

Sua sintaxe é bem simples:

```
SELECT  
    COUNT(coluna)  
FROM tabela;
```

Repare que, no exemplo ao lado, fizemos um COUNT da coluna contact_name da tabela customers, que retornou o total de 91 clientes.

Tudo perfeito, então, certo?

Sim, entretanto, tome cuidado...

```
SELECT  
    COUNT(contact_name)  
FROM customers;
```

Data output		Messages	Notifications
≡+	📄	▼	🗑️
	count	🔒	
	bigint		
1		91	

Pois...

Veja só este exemplo:

Fizemos o mesmo COUNT na tabela customers, mas, desta vez, informamos a coluna region.

Agora, foi retornado um total de apenas 31 clientes.

Por que isso aconteceu?

```
SELECT  
    COUNT(region)  
FROM customers;
```

Data output		Messages	Notifications
≡+	📄	▼	📋
		🗑️	📦
		🔒	⬇️
	count bigint		
1	31		

Porque a função **COUNT** não considera valores nulos em sua contagem.

Se você fizer uma consulta às colunas `contact_name` e `region`, verá que:

- A coluna `contact_name` não tem valores nulos.
- Já a coluna `region` possui valores nulos.

Assim, como o **COUNT** não considera valores nulos, as consultas retornaram valores diferentes.

```
237 SELECT
238     contact_name
239 FROM customers;
```

	contact_name character varying (30)
1	Maria Anders
2	Ana Trujillo
3	Antonio Moreno
4	Thomas Hardy
5	Christina Berglund
6	Hanna Moos
7	Frédérique Citeaux
8	Martín Sommer
9	Laurence Leblan
10	Elizabeth Lincoln
11	Victoria Ashworth
12	Patricio Simpson
13	Francisco Chang
14	Yang Wang
15	Pedro Afonso
16	Elizabeth Brown

```
247 SELECT
248     region
249 FROM customers;
```

	region character varying (15)
25	[null]
26	[null]
27	[null]
28	[null]
29	[null]
30	[null]
31	SP
32	OR
33	DF
34	RJ
35	Táchira
36	OR
37	Co. Cork
38	Isle of Wight
39	[null]
40	[null]
41	[null]

Para resolver essa questão dos valores nulos que são desconsiderados pela função COUNT, em vez de especificar qual coluna queremos contar, podemos simplesmente informar um * em seu lugar, conforme no exemplo ao lado.

Assim, não precisamos nos preocupar se a coluna informada possui valores nulos, uma vez que a função COUNT(*) faz a contagem de todas as linhas de uma tabela, contendo ou não valores nulos.

253	SELECT
254	COUNT(*)
255	FROM customers;
256	
Data output Messages Notifications	
≡+ ▼    	
	count bigint 
1	91

A função de agregação SUM faz a soma dos valores de uma coluna.

Sintaxe:

```
SELECT  
    SUM(coluna)  
FROM tabela;
```

No exemplo ao lado, fizemos a soma da coluna units_in_stock da tabela products, que retornou o total de unidades dos produtos em estoque.

```
SELECT  
    SUM(units_in_stock) AS "TotalEstoque"  
FROM products;
```

Data output		Messages	Notifications
≡+	📄	▼	🗑️
TotalEstoque		🗑️	📄
bigint		🗑️	📄
1	3119		

AVG, MIN, MAX

114

As funções de agregação AVG, MIN, MAX retornam, respectivamente: a média, o menor e o maior valor de uma coluna.

Sintaxe:

```
SELECT  
  SUM(coluna)  
FROM tabela;
```

```
SELECT  
  MIN(coluna)  
FROM tabela;
```

```
SELECT  
  MAX(coluna)  
FROM tabela;
```

```
SELECT  
  AVG(unit_price) AS preco_medio,  
  MIN(unit_price) AS menor_preco,  
  MAX(unit_price) AS maior_preco  
FROM products;
```

No exemplo ao lado, em relação à coluna unit_price da tabela products:

- Calculamos sua media (AVG);
- Descobrimos seu menor valor (MIN);
- Descobrimos seu maior valor (MAX).

Data output Messages Notifications			
      			
	preco_medio double precision	menor_preco real	maior_preco real
1	28.8338960920061	2.5	263.5

GROUP BY – Criando agrupamentos

115

O comando GROUP BY permite agrupar valores de acordo com uma coluna.

O GROUP BY é usado junto com funções de agregação (COUNT(), MAX(), MIN(), SUM(), AVG()) para agrupar valores de acordo com uma ou mais colunas.

Abaixo, vemos um exemplo do comando GROUP BY aplicado em conjunto com a função de agregação SUM para retornar a soma total de estoque (units_in_stock) por supplier_id.

```
SELECT
    supplier_id,
    SUM(units_in_stock)
FROM products
GROUP BY supplier_id;
```



Data output				Messages	Notifications
	supplier_id smallint		sum bigint		
1	29		130		
2	4		64		
3	10		20		
4	9		165		
5	7		110		
6	15		164		
7	6		98		
8	26		57		

GROUP BY – Criando agrupamentos

116

O comando GROUP BY combinado com o ORDER BY permite que a tabela agrupada seja também ordenada.

Veja agora como agrupamos a quantidade total de clientes por país, ordenando esse agrupamento pela contagem em ordem crescente:

```
SELECT
    country,
    COUNT(*)
FROM customers
GROUP BY country
ORDER BY COUNT(*);
```



	country character varying (15)	count bigint
1	Poland	1
2	Norway	1
3	Ireland	1
4	Portugal	2
5	Finland	2
6	Belgium	2
7	Denmark	2
8	Sweden	2
9	Austria	2
10	Switzerland	2
11	Argentina	3
12	Italy	3
13	Canada	3
14	Venezuela	4
15	Spain	5
16	Mexico	5
17	UK	7
18	Brazil	9
19	Germany	11
20	France	11
21	USA	13

GROUP BY, WHERE e HAVING – Filtros em agrupamentos

117

A combinação **GROUP BY + WHERE** nos permite criar filtros **antes** de agrupar uma tabela, para depois fazer o agrupamento de dados a partir de uma ou mais colunas dessa tabela.

Vamos a um exemplo:

Faça um agrupamento da quantidade total de clientes por país, considerando apenas os clientes cujo `contact_title` seja igual a “Owner”.

Neste caso, primeiro a gente vai precisar filtrar a tabela `customers` para retornar apenas os clientes que tenham o `contact_title = 'Owner'`. Para isso, usamos o **WHERE**.

Assim, podemos fazer uma contagem (`COUNT(*)`) apenas dos registros retornados (os clientes cujo `contact_title` seja igual a “Owner”), agrupando-os (**GROUP BY**) pelo país (`country`).

Ao lado, veja como fica a nossa consulta e seu resultado:

```
SELECT
  country,
  COUNT(*)
FROM customers
WHERE contact_title = 'Owner'
GROUP BY country;
```

Data output			Messages	Notifications
	country character varying (15)	count bigint		
1	Spain	1		
2	Mexico	3		
3	Switzerland	1		
4	Poland	1		
5	Germany	1		
6	Venezuela	2		
7	Denmark	1		
8	Norway	1		
9	Sweden	1		
10	USA	2		
11	France	3		

GROUP BY, WHERE e HAVING – Filtros em agrupamentos

118

Já a combinação **GROUP BY + HAVING** nos permite primeiro fazer o agrupamento de dados a partir de uma ou mais colunas de uma tabela, para **depois** criar filtros a partir desse agrupamento.

Vamos a um exemplo:

Faça um agrupamento da quantidade total de clientes por país, e retorne apenas os países que tenham mais de 10 clientes.

Aqui, primeiro vamos fazer uma contagem (COUNT(*)) do total de clientes, agrupando-os (GROUP BY) pelo país (country).

Feito isso, utilizamos o HAVING para extrair deste agrupamento criado somente aqueles países cuja contagem retornou um valor maior que 10.

Ao lado, veja como fica a nossa consulta e seu resultado:

```
SELECT
    country,
    COUNT(*)
FROM customers
GROUP BY country
HAVING COUNT(*) > 10;
```

Data output		Messages	Notifications
	country character varying (15)	count bigint	
1	USA		13
2	France		11
3	Germany		11

GROUP BY, WHERE e HAVING – Filtros em agrupamentos

119

Portanto, para saber quando utilizar o **GROUP BY** associado à cláusula WHERE ou à cláusula HAVING, você deve se perguntar:

Este filtro precisa ser feito na tabela que já existe ou no agrupamento que estou criando?

NA TABELA:

UTILIZE O **WHERE**
ANTES DO GROUP BY

SE SUA
RESPOSTA FOR:



NO AGRUPAMENTO:

UTILIZE O **HAVING**
DEPOIS DO GROUP BY

EXERCÍCIOS



1

Questão 1

A gerência deseja saber quantos fornecedores estão cadastrados no sistema para avaliar as possibilidades de novas parcerias.

Crie uma consulta que retorne a quantidade total de fornecedores. (Utilize a tabela `suppliers`)

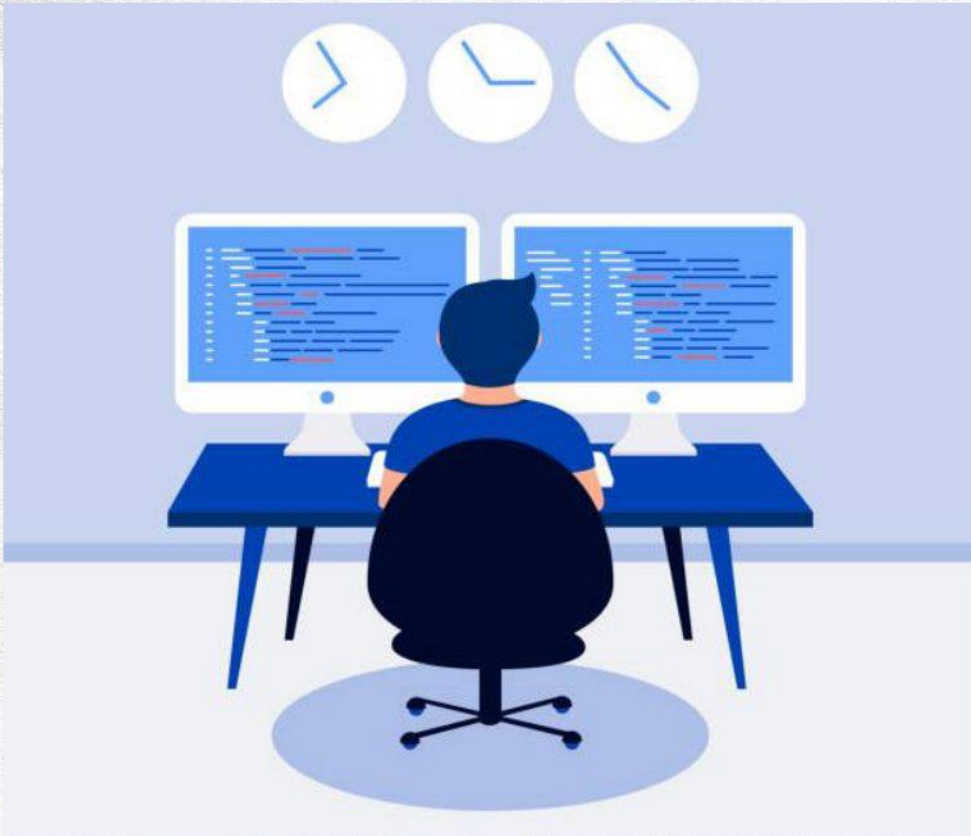
2

Questão 2

O gerente financeiro precisa saber o preço médio dos produtos no sistema para avaliar o custo médio dos itens vendidos.

Crie uma consulta para calcular a média de preço dos produtos. (Utilize a tabela `products`)

EXERCÍCIOS



3

Questão 3

A equipe de vendas deseja saber qual é o menor e o maior valor de frete já cobrado em pedidos para entender a variabilidade nos custos de entrega.

Crie a consulta que retorna esses valores. **(Utilize a tabela orders)**

4

Questão 4

O setor de Recursos Humanos está realizando um levantamento regional para identificar a quantidade de funcionários que residem na região "WA". Essa informação é fundamental para a organização de um evento corporativo local.

Desenvolva uma consulta que mostre a contagem total de funcionários nessa região. **(Utilize a tabela employees)**

EXERCÍCIOS



5

Questão 5

O setor de logística está avaliando o volume de produtos encomendados que são embalados em garrafas. Para isso, eles precisam saber o total de unidades desses produtos que estão em pedidos pendentes.

Crie uma consulta que retorne a soma das unidades encomendadas (`units_on_order`) de todos os produtos cuja descrição no campo `"quantity_per_unit"` mencione "bottles". (Utilize a tabela `products`)

6

Questão 6

O setor de vendas deseja saber quantos produtos estão disponíveis em cada categoria.

Crie uma consulta que retorne o ID da categoria e a quantidade de produtos associados a ela. Ordene o resultado pela contagem da quantidade de produtos em ordem decrescente. (Utilize a tabela `products`)

EXERCÍCIOS



7

Questão 7

O setor financeiro está analisando os custos logísticos relacionados ao envio de pedidos para diferentes países. Para isso, eles precisam saber o total acumulado de frete para cada país de destino, a fim de identificar quais regiões geram os maiores custos de transporte.

Elabore uma consulta que exiba o nome do país de envio e a soma total dos fretes, ordenando o resultado do maior para o menor valor. **(Utilize a tabela orders)**

8

Questão 8

A equipe de gestão de inventário está buscando uma forma de monitorar o estoque de cada categoria de produtos. Eles precisam entender quantos produtos existem em cada categoria e qual o total de unidades em estoque. Esse controle é essencial para que possam planejar reabastecimentos, identificar possíveis excessos ou faltas de estoque e tomar decisões informadas sobre estratégias de vendas e promoções.

Crie uma consulta que retorne o ID da categoria, a contagem de produtos e a soma do estoque por categoria. **(Utilize a tabela products)**

EXERCÍCIOS



9

Questão 9

A equipe de análise de vendas da empresa recebeu uma solicitação para avaliar o desempenho médio de preço dos produtos nas categorias de bebidas (`category_id = 1`) e laticínios (`category_id = 4`). O objetivo é entender a variação de preço dessas categorias para tomar decisões estratégicas sobre promoções e ajustes de preços.

Sua tarefa é criar uma consulta SQL que calcule o preço médio por unidade para as categorias de bebidas e laticínios, pois isso ajudará a equipe a tomar futuras decisões comerciais. **(Utilize a tabela `products`)**

10

Questão 10

A equipe de vendas da empresa está analisando o desempenho dos produtos para entender quais itens estão gerando maior volume de vendas. Para ajudar na tomada de decisões sobre estoque e estratégias de marketing, eles precisam obter um levantamento detalhado das quantidades vendidas de cada produto.

Crie uma consulta SQL que retorne a soma total de unidades vendidas (`quantity`) por produto, mas somente para os produtos que já alcançaram no mínimo 1000 unidades vendidas. A consulta deve exibir os produtos com o maior volume de vendas primeiro, para que a equipe de vendas possa priorizar os itens mais vendidos. **(Utilize a tabela `order_details`)**

GABARITO EXPLICATIVO

1

Questão 1

A gerência deseja saber quantos fornecedores estão cadastrados no sistema para avaliar as possibilidades de novas parcerias.

Crie uma consulta que retorne a quantidade total de fornecedores. (Utilize a tabela suppliers)

Explicação e Resultado

Para contarmos quantos fornecedores estão cadastrados no sistema, fazemos um **SELECT** na tabela **SUPPLIERS**. Como cada registro (linha) dessa tabela contém as informações de um fornecedor, podemos **contar sua quantidade de linhas**. Para isso, usamos a função de agregação **COUNT(*)**. Para que no resultado fique claro a que se refere essa contagem, podemos **fornecer um alias ao COUNT(*)**, chamando-a de **'total_fornecedores'**:

```
SELECT COUNT(*) AS total_fornecedores  
FROM suppliers;
```

Dessa forma, obtemos o número total de fornecedores cadastrados no sistema:

	total_fornecedores bigint	
1		29

GABARITO EXPLICATIVO

2

Questão 2

O gerente financeiro precisa saber o preço médio dos produtos no sistema para avaliar o custo médio dos itens vendidos.

Crie uma consulta para calcular a média de preço dos produtos. (Utilize a tabela **products**)

Explicação e Resultado

Para calcularmos o preço médio dos produtos no sistema, fazemos um **SELECT** na tabela **PRODUCTS**. A coluna **unit_price** contém os preços individuais de cada produto, e para calcularmos a média desses valores, utilizamos a função de agregação **AVG**. Para tornar o resultado mais claro, atribuímos um alias à função, chamando-a de **preco_medio_produtos**:

```
SELECT AVG(unit_price) AS preco_medio_produtos  
FROM products;
```

Dessa forma, obtemos o preço médio de todos os produtos cadastrados no sistema:

	preco_medio_produtos double precision 🔒
1	28.83389609200614

GABARITO EXPLICATIVO

3

Questão 3

A equipe de vendas deseja saber qual é o menor e o maior valor de frete já cobrado em pedidos para entender a variabilidade nos custos de entrega.

Crie a consulta que retorna esses valores. (Utilize a tabela **orders**)

Explicação e Resultado

Para descobrirmos o menor e o maior valor de frete já cobrado em pedidos, utilizamos a função **MIN** para identificar o menor valor e a função **MAX** para identificar o maior valor. Essas funções são aplicadas à coluna **freight** da tabela **ORDERS**. Para facilitar a leitura, fornecemos aliases aos resultados como **menor_frete** e **maior_frete**:

```
SELECT
  MIN(freight) AS menor_frete,
  MAX(freight) AS maior_frete
FROM orders;
```

Dessa forma, obtemos os valores mínimo e máximo cobrados em fretes, permitindo a análise de variabilidade nos custos de entrega:

	menor_frete real	maior_frete real
1	0.02	1007.64

GABARITO EXPLICATIVO

4

Questão 4

O setor de Recursos Humanos está realizando um levantamento regional para identificar a quantidade de funcionários que residem na região "WA". Essa informação é fundamental para a organização de um evento corporativo local.

Desenvolva uma consulta que mostre a contagem total de funcionários nessa região. (Utilize a tabela **employees**)

Explicação e Resultado

Para contarmos o número de funcionários que residem na região "WA", fazemos um **SELECT** na tabela **EMPLOYEES** e aplicamos um filtro com a cláusula **WHERE**, limitando a consulta à coluna **region** com valor igual a "WA". Utilizamos a função **COUNT(*)** para contar essas ocorrências e fornecemos um alias como **funcionarios_em_wa**:

```
SELECT COUNT(*) AS funcionarios_em_wa
FROM employees
WHERE region = 'WA';
```

Assim, obtemos a quantidade total de funcionários dessa região, permitindo o planejamento do evento corporativo:

	funcionarios_em_wa bigint
1	5

GABARITO EXPLICATIVO

5

Questão 5

O setor de logística está avaliando o volume de produtos encomendados que são embalados em garrafas. Para isso, eles precisam saber o total de unidades desses produtos que estão em pedidos pendentes.

Crie uma consulta que retorne a soma das unidades encomendadas (units_on_order) de todos os produtos cuja descrição no campo "quantity_per_unit" mencione "bottles". (Utilize a tabela products)

Explicação e Resultado

Para calcular o total de unidades encomendadas de produtos embalados em garrafas, fazemos um **SELECT** na tabela **PRODUCTS** e utilizamos a cláusula **WHERE** para filtrar os registros cuja coluna **quantity_per_unit** contém o texto 'bottles', usando o operador **LIKE** para tanto, passando como seu parâmetro '%bottles%'. Em seguida, somamos as quantidades encomendadas com a função **SUM** aplicada à coluna **units_on_order**. Para tornar o resultado mais claro, atribuímos um alias à função, chamando-a de **total_encomendado**:

```
SELECT SUM(units_on_order) AS total_encomendado
FROM products
WHERE quantity_per_unit LIKE '%bottles%';
```

Dessa forma, obtemos o total de unidades encomendadas desses produtos, atendendo à solicitação do setor de logística.

	total_encomendado 
1	120

GABARITO EXPLICATIVO

6

Questão 6

O setor de vendas deseja saber quantos produtos estão disponíveis em cada categoria.

Crie uma consulta que retorne o ID da categoria e a quantidade de produtos associados a ela. Ordene o resultado pela contagem da quantidade de produtos em ordem decrescente. (Utilize a tabela products)

Explicação e Resultado

Para saber quantos produtos existem em cada categoria, utilizamos a tabela **PRODUCTS**. Contamos os produtos com a função **COUNT(*)**, formando a coluna **quantidade_produtos**, e agrupamos os resultados pela coluna **category_id** usando **GROUP BY**. Para exibir as categorias com mais produtos primeiro, adicionamos a cláusula **ORDER BY** em ordem decrescente:

```
SELECT
    category_id,
    COUNT(*) AS quantidade_produtos
FROM products
GROUP BY category_id
ORDER BY quantidade_produtos DESC;
```

	category_id smallint	quantidade_produtos bigint
1	3	13
2	1	12
3	8	12
4	2	12
5	4	10
6	5	7
7	6	6
8	7	5
Total rows: 8 of 8 Query complete 00:00:		

Assim, obtemos uma lista das categorias e suas respectivas quantidades de produtos organizadas em ordem decrescente:

GABARITO EXPLICATIVO

7

Questão 7

O setor financeiro está analisando os custos logísticos relacionados ao envio de pedidos para diferentes países. Para isso, eles precisam saber o total acumulado de frete para cada país de destino, a fim de identificar quais regiões geram os maiores custos de transporte.

Elabore uma consulta que exiba o nome do país de envio e a soma total dos fretes, ordenando o resultado do maior para o menor valor. (Utilize a tabela orders)

Explicação e Resultado

Para calcular o custo total de frete por país de destino, utilizamos a tabela **ORDERS**. Somamos os valores da coluna **freight** com a função **SUM** e agrupamos os resultados pela coluna **ship_country**. Para facilitar a análise, ordenamos os resultados em ordem decrescente de fretes acumulados:

```
SELECT
    ship_country,
    SUM(freight) AS total_frete
FROM orders
GROUP BY ship_country
ORDER BY total_frete DESC;
```

	ship_country character varying (15) 🔒	total_frete real 🔒
1	USA	13771.285
2	Germany	11283.278
3	Austria	7391.5005
4	Brazil	4880.19
5	France	4237.8394
6	Sweden	3237.6003
7	UK	2954.27
8	Ireland	2755.24
Total rows: 21 of 21		Query complete 00:

Dessa forma, identificamos a soma total dos fretes para cada país de envio:

GABARITO EXPLICATIVO

8

Questão 8

A equipe de gestão de inventário está buscando uma forma de monitorar o estoque de cada categoria de produtos. Eles precisam entender quantos produtos existem em cada categoria e qual o total de unidades em estoque. Esse controle é essencial para que possam planejar reabastecimentos, identificar possíveis excessos ou faltas de estoque e tomar decisões informadas sobre estratégias de vendas e promoções.

Crie uma consulta que retorne o ID da categoria, a contagem de produtos e a soma do estoque por categoria. (Utilize a tabela `products`)

Explicação e Resultado

Para monitorar o estoque de produtos por categoria, fazemos um **SELECT** na tabela **PRODUCTS**. Contamos os produtos com a função **COUNT(*)** e calculamos o total de unidades em estoque com a função **SUM** aplicada à coluna **units_in_stock**. Agrupamos os resultados pela coluna **category_id**:

```
SELECT
  category_id,
  COUNT(*) AS qtd_produtos_categoria,
  SUM(units_in_stock) AS estoque_total
FROM products
GROUP BY category_id;
```

Assim, obtemos o total de produtos e unidades disponíveis por categoria, auxiliando o planejamento de inventário:

	category_id smallint	qtd_produtos_categoria bigint	estoque_total bigint
1	8	12	701
2	7	5	100
3	1	12	559
4	5	7	308
5	4	10	393
6	2	12	507
7	6	6	165
8	3	13	386
Total rows: 8 of 8 Query complete 00:00:00.080			

GABARITO EXPLICATIVO

9

Questão 9

A equipe de análise de vendas da empresa recebeu uma solicitação para avaliar o desempenho médio de preço dos produtos nas categorias de bebidas (`category_id = 1`) e laticínios (`category_id = 4`). O objetivo é entender a variação de preço dessas categorias para tomar decisões estratégicas sobre promoções e ajustes de preços.

Sua tarefa é criar uma consulta SQL que calcule o preço médio por unidade para as categorias de bebidas e laticínios, pois isso ajudará a equipe a tomar futuras decisões comerciais. (Utilize a tabela `products`)

Explicação e Resultado

Para calcularmos o preço médio de produtos nas categorias de bebidas (`category_id = 1`) e laticínios (`category_id = 4`), utilizamos a tabela `PRODUCTS`. Filtramos as categorias com a cláusula `WHERE` e o operador `IN`, e aplicamos a função `AVG` à coluna `unit_price`. Agrupamos os resultados pela coluna `category_id` para obter a média separadamente para cada categoria:

```
SELECT
    category_id,
    AVG(unit_price)
FROM products
WHERE category_id IN (1, 4)
GROUP BY category_id;
```

Dessa forma, a equipe de vendas pode analisar e comparar os preços médios das categorias selecionadas:

	category_id smallint 🔒	avg double precision 🔒
1	1	37.979166666666664
2	4	28.729999923706053

GABARITO EXPLICATIVO

10

Questão 10

A equipe de vendas da empresa está analisando o desempenho dos produtos para entender quais itens estão gerando maior volume de vendas. Para ajudar na tomada de decisões sobre estoque e estratégias de marketing, eles precisam obter um levantamento detalhado das quantidades vendidas de cada produto.

Crie uma consulta SQL que retorne a soma total de unidades vendidas (quantity) por produto, mas somente para os produtos que já alcançaram no mínimo 1000 unidades vendidas. A consulta deve exibir os produtos com o maior volume de vendas primeiro, para que a equipe de vendas possa priorizar os itens mais vendidos. (Utilize a tabela `order_details`)

Explicação e Resultado

Para identificar os produtos que alcançaram pelo menos 1000 unidades vendidas, utilizamos a tabela `ORDER_DETAILS`. Somamos as quantidades vendidas com a função `SUM` aplicada à coluna `quantity` e agrupamos os resultados pela coluna `product_id`. Filtramos os produtos com a cláusula `HAVING`, garantindo que apenas aqueles com 1000 unidades ou mais sejam incluídos. Ordenamos os resultados em ordem decrescente:

```
SELECT
    product_id AS id_produto,
    SUM(quantity) AS total_vendido
FROM order_details
GROUP BY product_id
HAVING SUM(quantity) >= 1000
ORDER BY total_vendido DESC;
```

	id_produto smallint	total_vendido bigint
1	60	1577
2	59	1496
3	31	1397
4	56	1263
5	16	1158
6	75	1155
7	24	1125
8	40	1103
9	62	1083
10	71	1057
11	2	1057
12	21	1016
Total rows: 12 of 12		Query compl

Assim, obtemos os produtos com maior volume de vendas, permitindo à equipe priorizar estratégias de marketing e estoque para os itens mais vendidos:

SQL

PostgreSQL
Apostila completa